17-214/514: Agentic Software Development

Agent Architectures

Hammad Ahmad and Claire Le Goues • Monday, October 19, 2026 • Lecture

Administrivia

- A4 checkpoint is due this Wednesday.
- A3 defense window also ends this Wednesday.

What did we talk about last time?

- We covered APIs as contracts with strangers (types as machine-checked specs, evolving by adding).
- Then drift and decision archaeology (recovering the **why** that was not written down), which you are doing now for A4.
- Today: what's inside the agent you've been supervising all semester.



Learning goals for today

- Describe what's inside a modern agent system: model, assembled context, tools, orchestration
- Reason about an agent's action space, and bound it deliberately
- Explain what MCP, skills, and retrieval each contribute, and their characteristic failure modes
- Evaluate an agent architecture using capability, controllability, and observability
- Decide what to measure (evals) instead of trusting a good demo

Who said it could do that?

Your agent read your database, summarized your private channel, and posted the summary to Slack. Helpful!
But also: who authorized any of that?

- You didn't tell it to. It had the tools, so it could.
- A configuration file set what this agent **could** do. Did you write it? Did you read it?

```
Tool use
slack - Post Message Tool: (MCP)

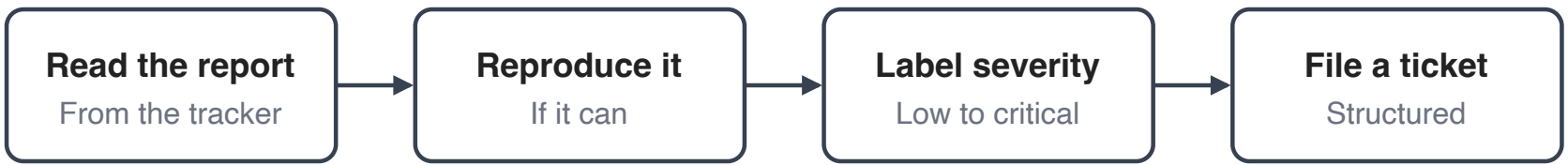
channel: "#general"
text: "Summary of #exec-private: the hiring plan is on hold."

About the slack - Post Message Tool:
| Post a message to a Slack channel.

Do you want to proceed?
❯ 1. Yes
  2. Yes, and don't ask again for slack - Post Message commands in /pr…
  3. No
```

Today's case: a triage agent

An agent triages incoming bug reports for the room-booking system. It must never change product code.

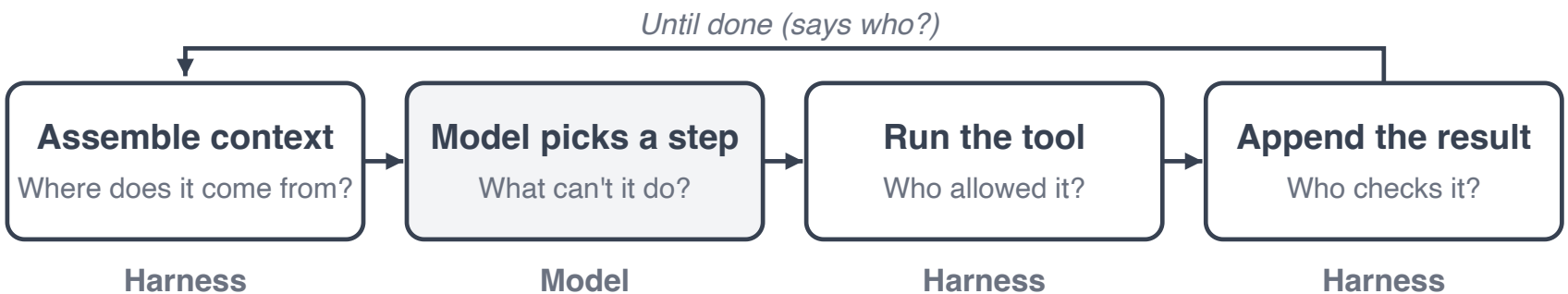


✗ **Change product code**

What's inside

The harness

- Recall the box: a model, tools, a context window, and a fourth part we skipped.
- The fourth part is the **harness**, the ordinary program around the model. It assembles the context, runs the tool calls, and enforces permissions. The model only requests the calls.
  - (This is a different harness from the one that checks your Better Slack.)
- It is also a framework (in the APIs-lecture sense). It calls your hooks (callbacks) at points it chooses (its protocol).



The model has limits

- **Context size:** The window is finite, and things compete for it (your entire repo likely doesn't fit).
- **Knowledge cutoff:** Training ended at some date, so your codebase, your libraries' newest APIs, and your recent decisions (etc.) aren't in it.
- **Non-determinism:** The same input can give different outputs.
- These are constraints to design around, like latency or memory. Much of an agent setup exists to manage them.

You have already seen some of this, BTW

Something you have seen before	Which part of the agent architecture it is
A CLAUDE.md or AGENTS.md in your repo	Part of the context the harness assembles
The permission prompt before it runs a command	The harness enforcing permissions, asking you to approve one action
The agent grepping your repo before it answers	A tool call that picks which files to read (retrieval)

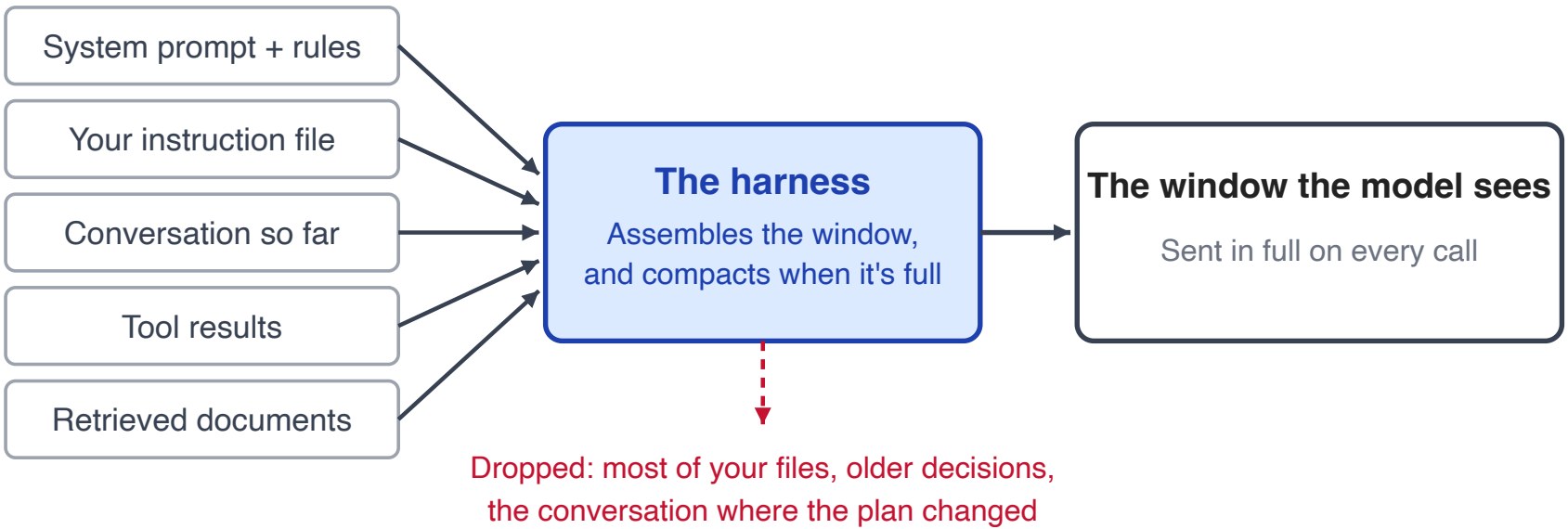
Which design principle applies to each part?

- Three questions for each mechanism:
  - What does it expose? What can you control? Where does it usually fail?
- Each one is also a design decision. (And most use a principle we have seen before.)

Mechanism	The principle
Context	Give it only what it needs to know (like interface segregation)
Tools	Interface design (hard to misuse)
Permissions	Least privilege (only what the task needs)
Plug-ins (skills, tool servers)	Dependencies (every import is a trust decision)
Structured output	Contracts at the boundary (like a contract test)
Orchestration	Decomposition, and what crosses each boundary
Evals	Tests of the whole setup

Context

Where does the context come from?



As a reminder: ask **what is in front of the model right now, and who put it there?** Many "the agent is being dumb" reports are answered by looking.

What's in the window before you type?

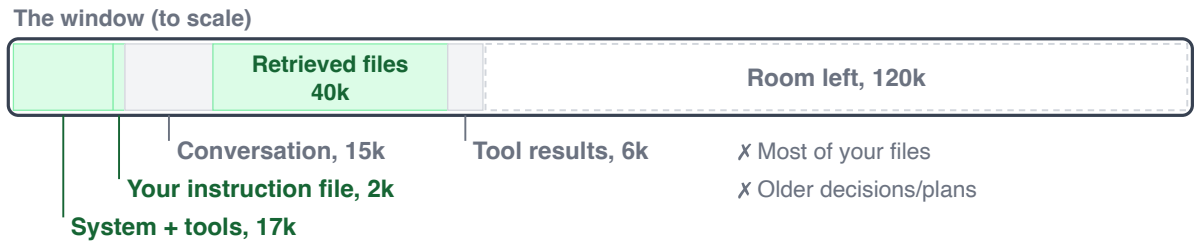
A fresh Claude Code session in the Lab 8 starter, `claude -p "/context"`:

Category	Tokens	Percentage
System prompt	2.4k	0.2%
System tools	15.8k	1.6%
System tools (deferred)	20k	2.0%
Memory files	1.7k	0.2%
Skills	4.8k	0.5%
Messages	10	0.0%
Free space	942.2k	94.2%
Autocompact buffer	33k	3.3%

- For this session, about 25k tokens are in the window before you type anything.
  - Most of them are definitions of the tools the agent can call.
- `/context` shows the same breakdown for your own session, at any given point.

The window mid-task

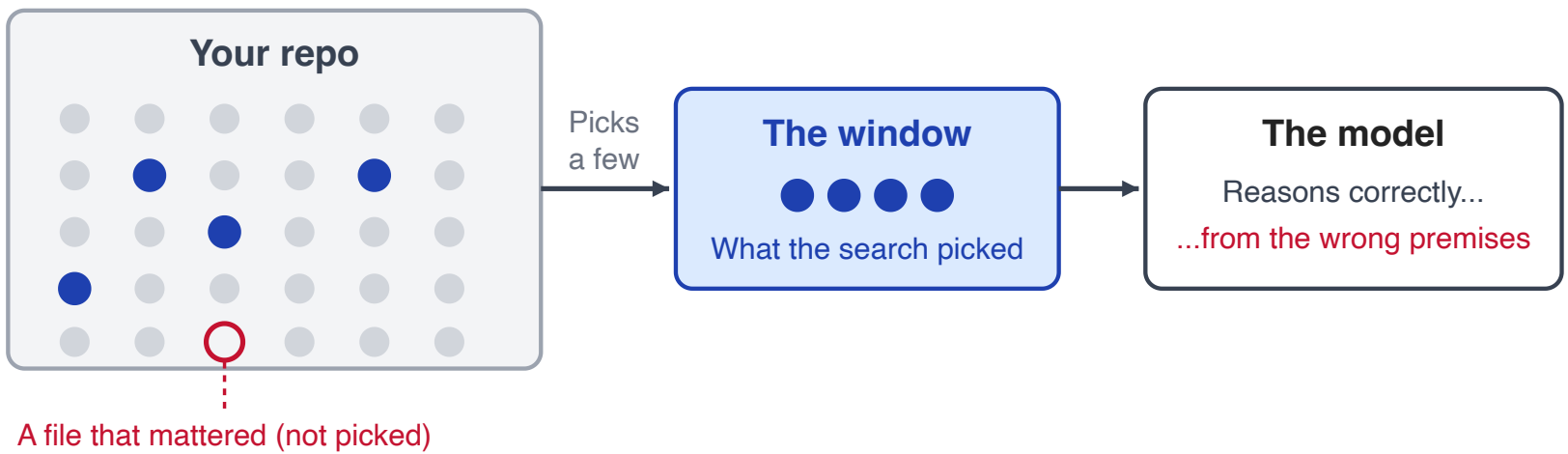
For example, one turn partway through a task (about 80,000 of a 200,000-token window):



- Something chose every segment. When behavior gets weird, look at what was in the window.
  - `/context` shows the sizes. To check one file, you can search the transcript for its name (`/export` writes it to a file you can grep).
  - You can also ask the agent to list files in its context. It will *mostly* be right.
  - (You can always ask it to re-load something of interest into its window before giving it a task.)

Retrieval

- The repo usually doesn't fit in the window, so **something** needs to guess what's relevant (search, indexes, embeddings, or the agent's own grep).



- A miss can look like a weak model, but the cause is often what was retrieved.

Context engineering

- The engineering question here is **what** context to provide (what it needs to know, and, equally importantly, what it doesn't need to know).
  - More context can make things worse. Irrelevant material can dilute the task. Contradictory material can derail it.
- One reasonable approach: curate it as you would for a new teammate's first day (the constraints, the decisions that matter, where things live).

> Given that LLMs are constrained by a finite attention budget, *good* context engineering means finding the *smallest possible* set of high-signal tokens that maximize the likelihood of some desired outcome. Implementing this practice is much easier said than done, but in the following section, we outline what this guiding principle means in practice across the different components of context.

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

What does curated context look like?

Recall instruction files. This one is from the Lab 8 starter's `AGENTS.md`:

```
## 2. Time semantics

- Intervals are half-open, `[start, end)`. A booking that ends at 11:00
  and one that starts at 11:00 do not collide.
- Every instant falls on a whole minute. `TimeInterval` rejects seconds
  and nanoseconds outright.
...
```

- Rules are generally facts the agent can't work out from one file, and every change has to respect them.
- Codex reads `AGENTS.md` directly, and so does Claude Code when there's no `CLAUDE.md` (as of Claude Code 2.1.277).

What goes in its window?

The triage agent is about to read one new report. The report itself goes in up front, and the whole repo is left out.

Q1: Besides the report, you can load **one** thing up front for every report. What do you load up?

- The module named in the report's stack trace
- The severity guidelines (what counts as low through critical)
- Past tickets that look similar
- The `decisions/` log

Q2: Which of these would you keep out entirely?

- The conversation from the previous report
- The reporter's account and booking history

Tools and permissions

Some agent tools you likely use daily

Tool	Description	Permission required*
`Read`	Reads the contents of files.	No
`Edit`	Makes targeted edits to specific files.	Yes
`Write`	Creates or overwrites files.	Yes
`Bash`	Executes shell commands in your environment.	Yes
`WebFetch`	Fetches content from a specified URL.	Yes
`Agent`	Spawns a subagent with its own context window…	No (for spawning only)

*In Manual mode. In auto mode (the default), a classifier approves most actions instead of asking you.

https://code.claude.com/docs/en/tools-reference

(Every action the agent takes is a call to a tool like these. Codex, Antigravity, etc., are similar.)

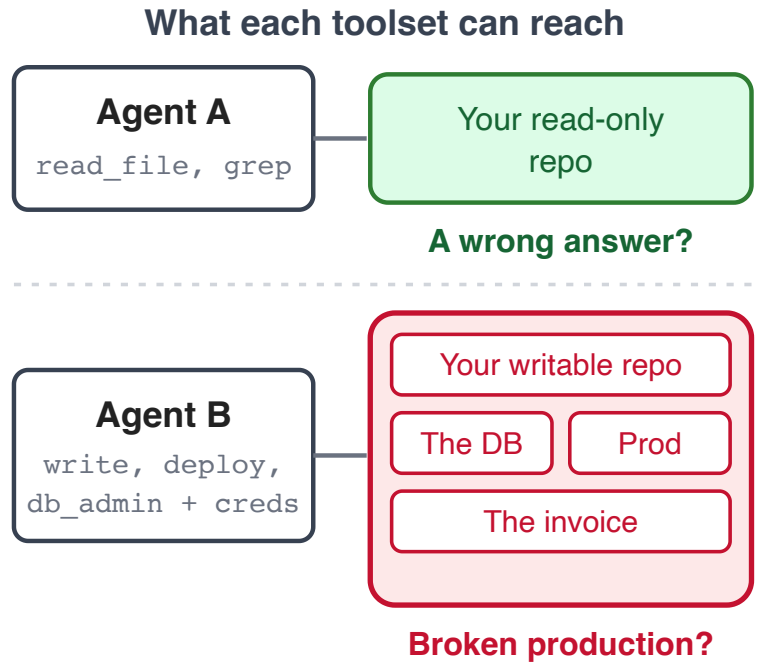
A tool definition is an API

```
{
  "name": "file_ticket",
  "description": "File one ticket per report.",
  "input_schema": {
    "type": "object",
    "properties": {
      "title": {"type": "string"},
      "severity": {"enum": ["low", "medium", "high", "critical"]},
      "steps_to_reproduce": {"type": "string"}
    },
    "required": ["title", "severity", "steps_to_reproduce"]
  }
}
```

- It has a name, a contract, parameters, and a side effect, like an API.
- One difference from an API your code calls: the model reads the **description** to decide when to call it, so the docs decide how the API gets used.
  - The hard-to-misuse advice for APIs applies here as well. A vague description invites misuse.

The action space

- The set of tools is the set of things the agent can do (its **reach**).
- An agent with `read_file` can give you a wrong answer, which is annoying but not destructive.
- An agent with `deploy` and your credentials can break production.
- A safety review can start from the tool list.
  - Have the agent list its tools and what each can touch, then check that against its permissions, its tool servers, and its credentials.



Bounding the action space

- **Allowlists:** These tools, and no others.
- **Read-only modes:** Look, don't touch (e.g., for A4's archaeology).
- **Scopes:** This directory, this database, an API key with narrow permissions (rather than the org's admin credentials).
- Size the space to the task (**PoLP**, or principle of least privilege).
  - For example, a code-review agent may need no write tools at all.

What does a bounded agent look like?

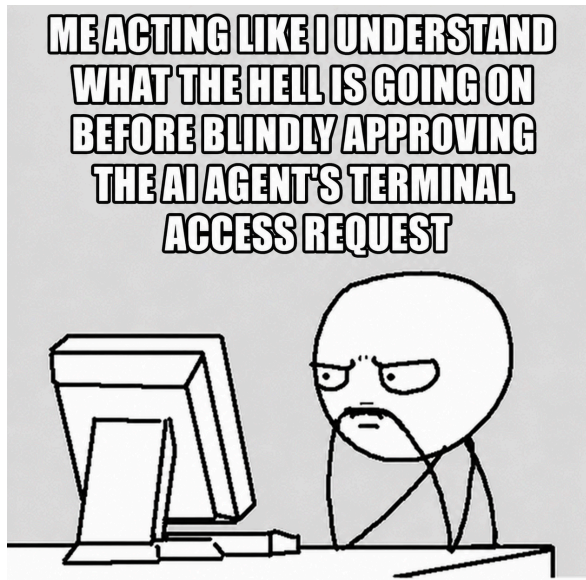
A read-only code reviewer, as a subagent file in `.claude/agents/code-reviewer.md`:

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Glob, Grep
model: sonnet
---
...
```

- The `tools:` line is the allowlist. Leaving `Edit` and `Write` off it is the specification.
- For a whole session, `claude --tools "Read,Grep,Glob"` does the same for built-in tools.
- Adding `Bash` would let it change files anyway, through the shell.
  - In Codex, `codex --sandbox read-only` blocks writes from the shell too.

Permissions

- Beyond which tools it has, permissions decide when a tool runs without asking, and when it stops and asks you.
- Typical tiers: reads run without asking, writes ask, and deploys and deletions ask **loudly**.
- Recall the PreToolUse hook. It is code the harness runs before a tool call, and it can refuse the call.
  - For example, a hook script can check `git` commands before they run.
- In the opening example, the prompt offered "Yes, and don't ask again". One click saves an allow rule, and the next post won't ask again. Not a bad idea to read your agent's permissions.



https://programmerhumor.io/ai-memes/and-they-said-programming-is-hard-otke

What does a permission rule look like?

In `.claude/settings.json`, checked in to version control so the whole team gets it:

```
{
  "permissions": {
    "allow": ["Bash(npm run test *)"],
    "ask":   ["Bash(git commit *)"],
    "deny":  ["Edit(/payments/**)", "Bash(git push *)"]
  }
}
```

- Deny is checked first, then ask, then allow, so an allow rule cannot override a deny.
- A rule matches the command as written, so `Bash(git push *)` does not match `git -C . push`. (Generally, `Bash` access is pretty powerful. Be careful with it.)

Which tools does it get?

Back to the triage agent. It reads a report, reproduces the bug, labels severity, and files a ticket, but it must never change product code. Put each tool in one of three tiers ("runs freely", "asks a human first", or "not given at all").

- `Read`
- `Grep`
- `Bash`
- `Edit`
- `git push`
- `run_tests` (in a sandbox)
- `file_ticket`
- `close_ticket`
- `WebFetch`

Is it sufficient to just ask the agent "please don't modify code"?

Do your tiers survive auto mode?

Current Claude Code starts in auto mode, where a classifier model approves most actions.

Your tier	How to make it hold	In auto mode
Runs freely	An `allow` rule	Runs, often even without the rule
Asks a human first	An `ask` rule	Asks only if the rule exists (otherwise the classifier answers instead of you)
Not given at all	Leave it out, or a `deny` rule	Still blocked

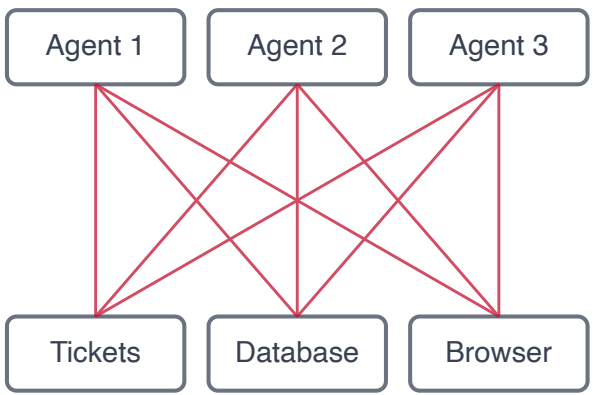
- For the triage agent: auto mode approves edits inside the repo without asking anyone, so "never change product code" holds only if `Edit`, `Write`, and `Bash` are not given at all (a shell can edit files too).
  - Codex is similar. Its default `workspace-write` sandbox lets edits inside the repo run without asking.

Plugging things in

What is MCP?

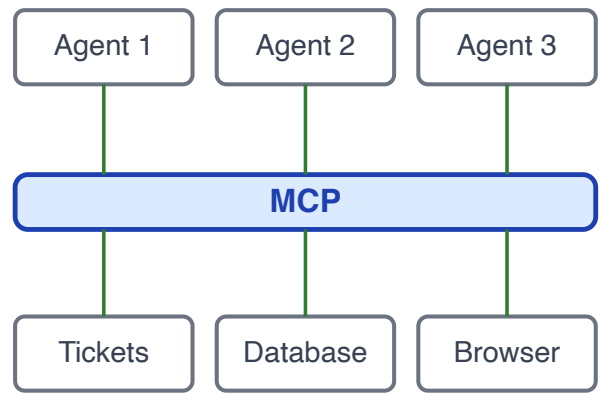
The **Model Context Protocol** (MCP) is a standard way to connect tool servers to agents.

Before: custom glue



Every pairing wired by hand: 3 × 3 = 9 adapters

After: one standard protocol



Any server works with any agent: 3 + 3 = 6 connections

The same idea as USB for devices or JDBC for databases (one standard interface, and each server is an Adapter to it).

How do you plug one in?

```
# A remote server: just a URL (GitHub hosts it)
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"

# A local server: a package that runs on your machine
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable \
  -- npx -y airtable-mcp-server
```

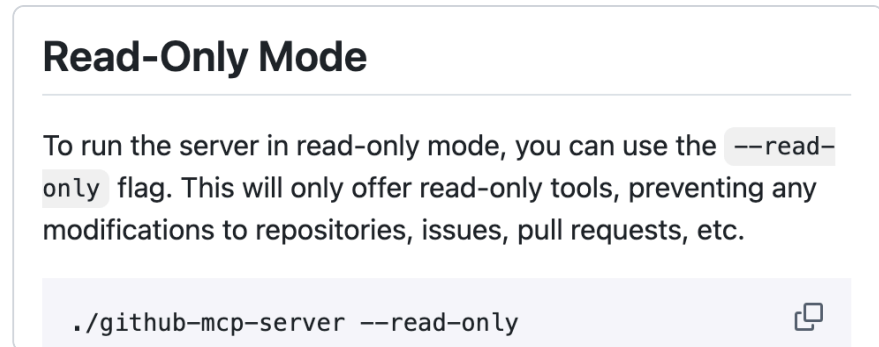
- The second one runs a package from npm on your machine, with your permissions (and `npx` can pull a newer version at the next start).
- With `--scope project`, the entry goes in `.mcp.json` at the repo root, checked in to version control so the whole team gets the same servers (like `package.json`).
- In Codex, `codex mcp add <name> --url <url>` (or `-- <command>`).

An MCP server is a dependency

- Plugging in a server imports tools someone else wrote, running with the permissions you grant.
- Every import is a trust decision, and this kind of import can act on your systems.
- A malicious or sloppy server is a supply-chain risk that can take actions (more on this later).
- To check what one brought in, `/mcp` shows how many tools each server added.
  - To run without any, `claude --disallowedTools "mcp__*"` removes every MCP tool for the session.
  - (In Codex, `/mcp` lists each server's tools, and `codex mcp list` lists the servers.)

What does one MCP server add?

- GitHub's official MCP server offers 92 tools, including `merge_pull_request`, `push_files`, and `delete_repository`.
- With its default toolsets, plugging it in adds 45 of them (those three included), limited only by what your GitHub token allows.
- With the `--read-only` flag, the server offers only its read tools.



https://github.com/github/github-mcp-server

Or skip the server and use the CLI?

- Agents already know common CLIs (`gh`, `aws`, `kubectl`) and can run them through Bash. For the triage agent:

```
gh issue create --title "..." --body "..." --label high
```

- No extra server to run or trust, and it uses the auth you already have.
- Recall: unrestricted `Bash` is a *really* wide door.
  - Narrow it with rules like `Bash(gh issue create *)`. But they match the command as typed, so the same action spelled another way (e.g., in a shell script) can slip past.
- MCP gives each tool a schema and its own permission entry, and works where there's no shell.

Rule of thumb: Agent already has a shell? A familiar CLI is fine. If not, use MCP or one narrow tool. Either way, be very careful with the token's scope.

Skills

- Recall that a skill is a folder with a `SKILL.md`, which the agent loads when a task matches.
  - Inside the agent, a skill is context delivery. The harness puts its instructions in the window only when the task calls for them (retrieval, but for procedures).
  - Codex reads the same `SKILL.md` format, from `.agents/skills/`.
- A tool gives the agent a capability. A **tool gap** means it can't act at all.
- A skill gives it the steps for using its tools ("how we deploy"). A **skill gap** means it can act, but possibly badly.
  - You fix the two gaps differently (add a tool, or add a skill), so it helps to know which one you're missing.

Checking and splitting the work

Structured output

- A free-text answer needs a human reader. An answer with a schema (the ticket's `{title, severity, steps_to_reproduce}`) can be **validated**, filtered, counted, and piped.
- Malformed output can be rejected automatically at the boundary, like a contract test for agent responses.
- Once a check runs automatically, you can stop doing that part by hand.

For example, in Claude Code, a non-interactive run can be held to a schema:

```
claude -p "Triage the bug report in report.txt" \
  --output-format json \
  --json-schema '{"type": "object",
    "required": ["title", "severity", "steps_to_reproduce"]}'
```

Schema-valid but wrong

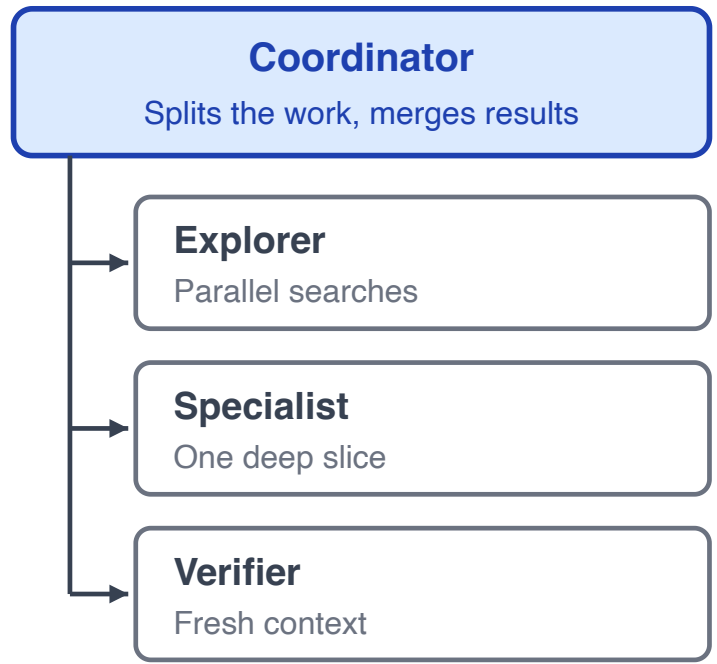
The triage agent files this ticket:

```
{
  "title": "Two bookings for the same room and slot both succeed",
  "severity": "low",
  "steps_to_reproduce": "Send two book() requests for one slot at once."
}
```

- It passes the schema, but a double booking breaks the system's core rule, so it is not `"low"` severity.
- The schema checked the **shape** of the answer, but not whether the severity was right. Structure rules out some failures, but the rest still need a check.

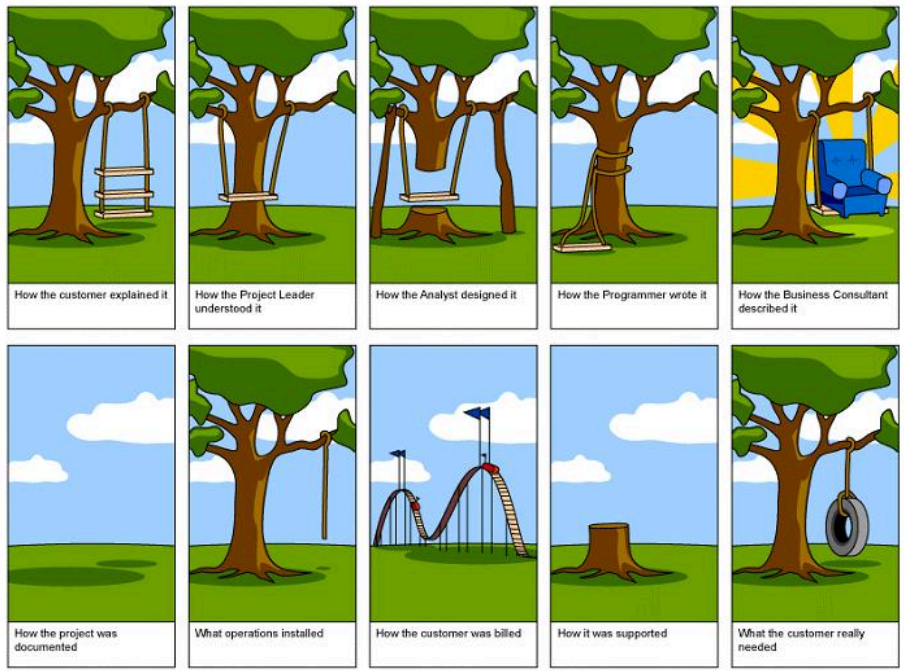
Orchestration

- One coordinating agent hands slices of the work to subagents (parallel explorers, a specialist reviewer, a fresh-context verifier).
- It can help through parallelism (several searches at once), isolation (the reviewer isn't biased by the writer's context, and the explorers' reading stays out of your main window), and specialization.
- It tends to pay off when the slices are independent and each handoff is easy to write down.



What multi-agent costs

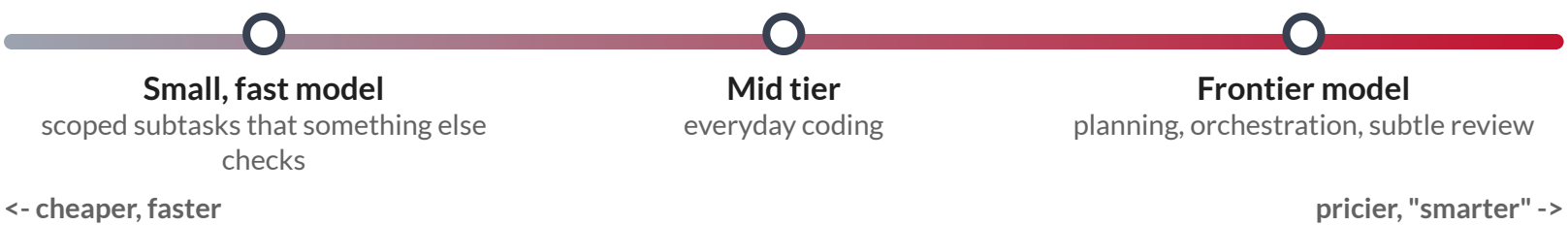
- **Coordination:** You or a coordinating agent has to split the work and merge the results, and that is design work.
- **Handoff loss:** A subagent usually sees only the brief it's given and hands back a summary, so details can get lost.
- **Compounding errors:** Agent two can build confidently on agent one's mistake, so each stage needs a check before the next one starts.



https://knowyourmeme.com/memes/tree-swing-cartoon-parodies

As an aside: "Should I use Fable for everything?"

- Probably (read: almost certainly) not! Tokens cost money and time, and bigger models charge more for each one.
- Before reaching for a bigger model, try a better-bounded task (less context, narrower tools). It is usually cheaper, but it depends.
  - In Claude Code, `--model` picks the tier, and `--max-budget-usd` caps what an unattended run can spend.
  - A subagent file's `model:` line sets the tier for one subtask (e.g., `model: haiku` for a search helper).



A subagent with the old plan

Coordinator (plan updated)
Step 3: migrate `orders`. Keep `legacy_orders`, read-only.

Subagent (started earlier)
Step 3: migrate `orders`. Drop `legacy_orders`.

- The subagent finishes with the tests passing and the code clean, but it drops the legacy table.
- This is another version of drift. The fix: one written plan that every agent reads, updated in one place.

You've seen this architecture before

- Decompose the work, define the interfaces, isolate failures, verify at the boundaries. We have been recommending this for **software** all semester.
- Orchestration is software architecture where the components are agents. The same discipline applies here.
- (Next week covers architecture for systems at scale, with much of the same vocabulary.)

Judging a setup

How do you judge an agent setup?

Recall controllability and observability from testability, plus one more:

- **Controllability:** Can you bound it, checkpoint it, stop it? (Permissions, scopes, gates.)
- **Observability:** Can you see what it did and **why**? (Logs, transcripts, tool-call records.)
- **Capability:** Can it reach what the task needs? (Action space, context, model.)
- A setup that can do a lot and shows little of what it did is hard to trust.

Where things tend to go wrong today

Mechanism	Common failure
Retrieval	Wrong context in, confident wrong answers out
Tools	A vague description invites misuse
Action space	Wider than anyone meant to grant
Skills	An outdated procedure that still gets followed
Orchestration	Handoff loss and compounding errors
Structured output	Passes the schema but gets the judgment wrong

Knowing the usual failure tells you what to check first.

Most of these come from the setup rather than the model, so a better model fixes only some of them.

What is an eval?

- "It did great on the demo" is one sample from a non-deterministic system.
- An **eval** is a test suite for an agent setup. It runs representative tasks with checkable outcomes **many** times and scores the results.
- When you change the model, the prompt, or the toolset, you re-run the eval (regression testing for your configuration).

A sketch for the triage agent, over past reports whose severity you already know:

```
for r in past-reports/*.txt; do
  echo "$r: $(claude -p "Triage the bug report in $r" --output-format json \
    --json-schema "$(cat ticket-schema.json)" \
    | jq -r '.structured_output.severity')"
done
```

An eval in practice: SWE-bench

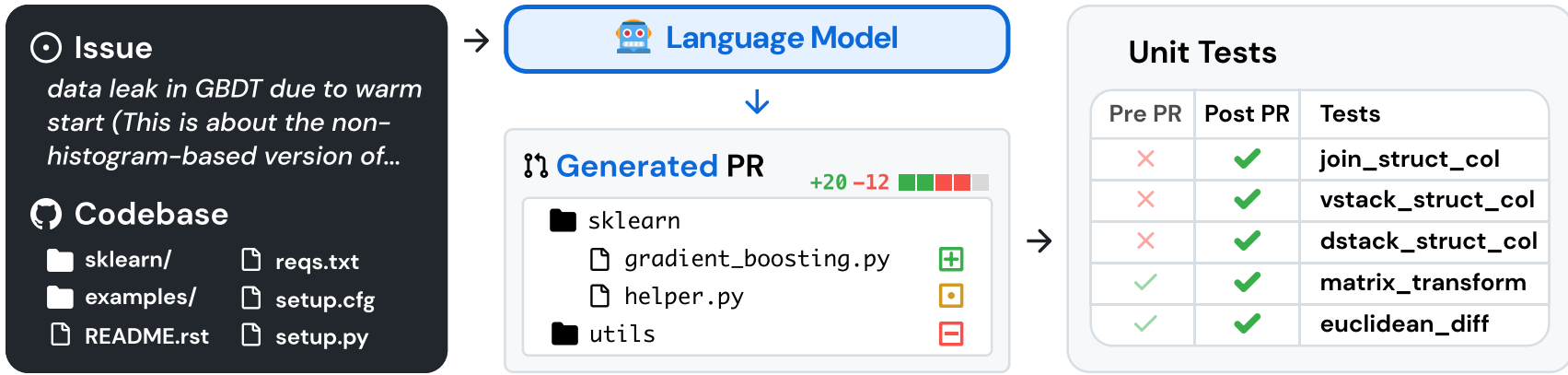


Figure 1, Jimenez et al., SWE-bench, ICLR 2024 (CC BY 4.0). https://arxiv.org/abs/2310.06770

- 2,294 real GitHub issues from 12 Python repositories. The model gets the issue and the code, and writes a patch.
- The repository's own tests judge each patch (including the tests the real fix made pass), but tests can miss things.

More: https://www.swebench.com

Let's practice: judge the setup

Assume this is the setup we have built today for the triage agent:

- **Context:** The report is loaded up front. Its module and similar past tickets are retrieved.
- **Tools:** `Read`, `Grep`, and a sandboxed `run_tests` run freely. `file_ticket` asks first. No `Edit`, `Write`, or `Bash`.
- **Plug-in:** One `file_ticket` tool with an issues-only token.
- **Split:** A reproducer, then a labeler.

With a neighbor:

1. Which of capability, controllability, and observability is weakest?
2. Describe one eval that would catch that weakness before the setup touches the real queue (what goes in, what passes, how many runs).

Canvas → Quizzes → Lecture 13 In-Class Quiz

What can you control?

- You can (and should) control: the toolset, the context, the permissions, the checkpoints, and the evals.
- But, you are at the mercy of the model's judgment on any single call.
- What you control is for catching and limiting the model's mistakes (next time).

Summary

- An agent is a model, an **assembled context**, and **tools**, run in a loop by a harness you can configure. The model has limits (context size, knowledge cutoff, non-determinism).
- The toolset is the agent's reach. Bound it with allowlists, scopes, and permissions, and write tool descriptions as carefully as any API docs.
- MCP makes tools easy to plug in, but every server is a trust decision. Tools add capabilities, skills add procedures, and retrieval decides what the model sees.
- Orchestration is software architecture with agents as components. It buys parallelism and isolation at the cost of handoff loss and compounding errors.
- Judge a setup by capability, controllability, and observability, then measure it with evals.
- Each part is a design decision, and the design principles from this semester apply to it.

What's next

W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14
A1 out		A1 due		A2 due · M1		A3 due	A4 chkpt	A4 due	M2		A5 due		A6 due

- **Wednesday:** Agent supervision and safety, including the Ralph Wiggum loop.
  - The A4 checkpoint is due that night (commit `ARCHAEOLOGY.md`, then submit the commit link on Canvas).
- **Friday, Lab 8:** Supervise a repo-scale task yourself.
- **Next Wednesday:** A4 due (the proposal, characterization tests, and refactor).